RELATÓRIO DE CHECK POINT

Projeto Sistemas Embarcados com ESP32 | Validação da evolução do projeto

Disciplina	Sistemas Embarcados – CESAR School
Professores	Bella Nunes | Jymmy Barreto
Data do Check Point	17/11/2026
Grupo (nº)	
Título do projeto	SolarSense – monitoramento não invasivo de sistemas fotovoltaicos
Resumo do projeto	Sistema IoT com ESP32 que mede irradiância, temperatura, umidade, pressão e particulados junto ao sistema fotovoltaico e publica as leituras via MQTT. Um back-end cruza esses dados com a geração do inversor e uma previsão de energia, e o dashboard mostra se a usina está produzindo o esperado.
Representante do grupo	Kauan Novello

Integrantes

Nº	Nome completo	Usuário no GitHub	Responsabilidade no projeto
1	Kauan Novello	kauan-novello	Projeto individual: firmware, circuito, back-end, front-end e infraestrutura
2			
3			
4			
5			
6			
7			

Como preencher

Substitua os textos cinza entre colchetes [ ], marque as opções com e insira imagens nas áreas pontilhadas. Seja objetivo: o foco é mostrar o quanto o projeto evoluiu até aqui. O painel da última página é de uso dos docentes.

1. Prototipagem

1.1 Uso dos sensores/atuadores

Orientação: *Mantenha apenas os componentes que o grupo já está usando e acrescente os que não constam na lista.*

Componente	Tipo	Função no projeto	Pino(s) / GPIO	Situação		
ESP32	Controlador	Lê os sensores e publica via MQTT (Wi-Fi). Botão BOOT (3 s) reabre o portal de configuração.	GPIO 0 (botão BOOT)	☒ OK	Parcial	Pendente
LDR	Sensor	Mede a irradiância (luz incidente)	GPIO 36 (ADC)	☒ OK	Parcial	Pendente
BME280/BMP280	Sensor	Mede temperatura, umidade e pressão atmosférica	GPIO 21 (SDA), 22 (SCL)	☒ OK	Parcial	Pendente
DHT11	Sensor	Mede temperatura e umidade do ar ambiente	GPIO 4	☒ OK	Parcial	Pendente
ZPH02	Sensor	Mede a concentração de material particulado (poeira sobre os painéis)	GPIO 16 (RX)	☒ OK	Parcial	Pendente

1.2 Organização do circuito

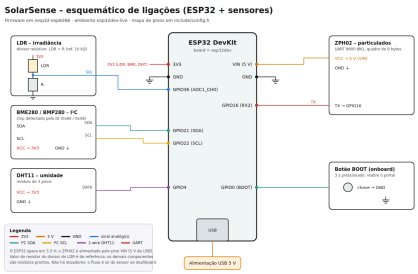


Figura 1 – Esquemático

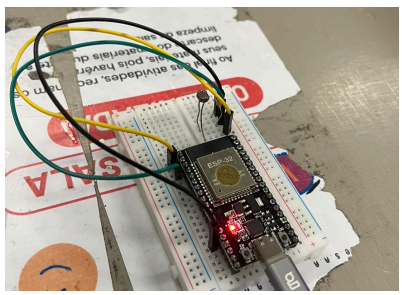


Figura 2 – Montagem física

Arquivo no repositório	/schematics/solarsense-esquematico.svg (e .png)
Circuito testado	☒ Cada componente testado individualmente Circuito completo testado
Situação	Concluído ☒ Em andamento Não iniciado
Próximos passos	Fixar as ligações da protoboard, que hoje se soltam com facilidade, fotografar a montagem final e testar o circuito completo (ambiente esp32dev-live) com a placa conectada ao broker autenticado.

2. Artefatos de Desenvolvimento

2.1 Códigos-fonte

Artefato	Tecnologias / linguagem	Caminho no repositório	Progresso	Situação
Firmware	C++, Arduino/PlatformIO, PubSubClient, WiFiManager, ArduinoJson	/esp32-esp8266	80 %	Concluído ☒ Em andamento Não iniciado

Back-End	Python, FastAPI, SQLAlchemy, Alembic, PostgreSQL, paho-mqtt	/applications	85 %	Concluído ☒ Em andamento Não iniciado
Front-End	TypeScript, React, Vite, Tailwind CSS, shadcn/ui	/applications	75 %	Concluído ☒ Em andamento Não iniciado

O que já funciona	Leitura dos 4 sensores; firmware com tarefas FreeRTOS, fila e mutex; portal de configuração; MQTT autenticado; ingestão e gravação no PostgreSQL; cadastro, login e pareamento; coleta do inversor Sungrow; dashboard com geração real × esperada, previsão e leituras ao vivo; alerta de superaquecimento por e-mail; testes e CI.
Próximos passos	Validar os sensores reais na bancada (esp32dev-live), testar o botão BOOT, instalar o protótipo junto à usina, refinar o dashboard e concluir o relatório técnico em /docs.

3. Arquitetura do Projeto

3.1 Fluxo de comunicação

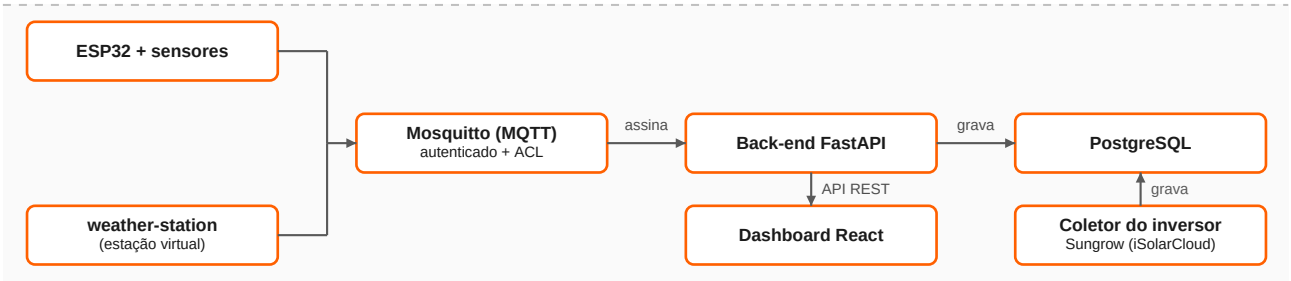


Figura 3 – Arquitetura e fluxo de comunicação

Descrição do fluxo	A ESP32 lê os sensores e publica um JSON no broker Mosquitto, autenticando com usuário e senha. O back-end assina o tópico, valida o payload, associa a leitura ao sistema pareado e grava no PostgreSQL. Um coletor separado grava os dados do inversor Sungrow no mesmo banco. O dashboard consulta o back-end pela API REST e mostra as leituras, a geração e a previsão. Não há atuadores: o fluxo é só do sensor para o dashboard.

Tópico MQTT	Publisher	Subscriber	Payload (formato / exemplo)
solarsense/{MAC}/readings	ESP32 (usuário device)	Back-end (usuário backend)	{"timestamp":"2026-11-12T13:00:00Z","irradiance":78.5, "ambient_temp_c":29.4,"ambient_humidity_pct":62.0, "pressure_hpa":1011.8,"particulate_pct":1.2}

Situação	Concluído ☒ Em andamento Não iniciado
Próximos passos	Validado em 13/11: a placa publica no broker autenticado e o back-end grava as leituras (valores simulados). Falta testar com os sensores reais e usar IP fixo em redes sem mDNS.

3.2 Infraestrutura (local/nuvem)

Broker MQTT	☒ Local (PC / WSL / contêiner / Raspberry Pi) ESP32 Nuvem: [ ]
Back-end e dashboard	☒ Local Nuvem Detalhes: Docker Compose: Mosquitto, PostgreSQL, back-end FastAPI, coletor do inversor, MLflow e Airflow; dashboard em Vite.
Serviços de nuvem	Não se aplica. A única dependência externa é a API iSolarCloud da Sungrow, que fornece os dados do inversor.

Situação	☒ Concluído ☐ Em andamento ☐ Não iniciado
Próximos passos	Avaliar TLS no broker e a hospedagem do back-end para acesso fora da rede local.

4. Documentação (Git)

URL do repositório	https://github.com/kauan-novello/Projeto-Embarcados-CesarSchool-CSBC
Visibilidade	☒ Público ☐ Privado (precisa ser público)
Gestão de tarefas	☐ GitHub Projects ☒ Issues ☐ Não utilizado
Commits do grupo	☒ Todos os integrantes já fizeram commits Total aproximado: 22

Item da estrutura	Situação			Observações
README.md	☒ Completo	☐ Parcial	☐ Ausente	Arquitetura, como rodar, variáveis de ambiente, firmware e segurança
/docs	☐ Completo	☒ Parcial	☐ Ausente	Relatório técnico em elaboração
/applications	☒ Completo	☐ Parcial	☐ Ausente	Back-end, front-end, mockup, coletor do inversor e estação virtual
/esp32-esp8266	☒ Completo	☐ Parcial	☐ Ausente	Firmware PlatformIO com três ambientes (sensores reais, ao vivo e bancada)
/schematics	☒ Completo	☐ Parcial	☐ Ausente	Esquemático em SVG e PNG

5. Aplicação

5.1 Dashboard

Tecnologia	React + TypeScript + Vite + Tailwind CSS (shadcn/ui)
Dados exibidos	Última leitura dos sensores (irradiância, temperatura, umidade, pressão, particulados), status do dispositivo, geração real do inversor × energia esperada pelo modelo, previsão dos próximos dias, histórico horário e alertas de superaquecimento.
Atualização em tempo real	Sim ☒ Parcial Ainda não
Envia comandos aos atuadores	Sim ☒ Não Previsto
Situação	Concluído ☒ Em andamento Não iniciado
Próximos passos	Atualização automática das leituras ao vivo e ajustes visuais nas telas de histórico.

5.2 Protótipo do Front-End

Ferramenta e link	Protótipo navegável em React (dados fictícios), em /applications/frontend-mockup do repositório.
Telas previstas	Login e cadastro (acesso do usuário); Lista de sistemas (usinas cadastradas e status); Cadastro/edição de sistema; Pareamento (vincular a ESP32 ao sistema); Dashboard (leituras ao vivo, geração real × esperada, previsão e alertas); Painel do modelo (métricas da previsão).

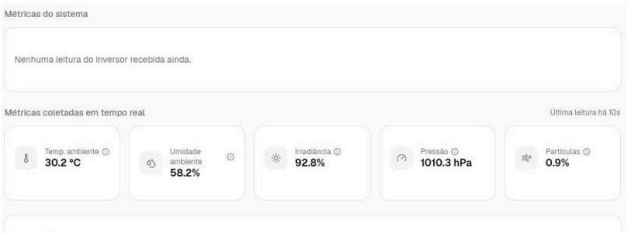


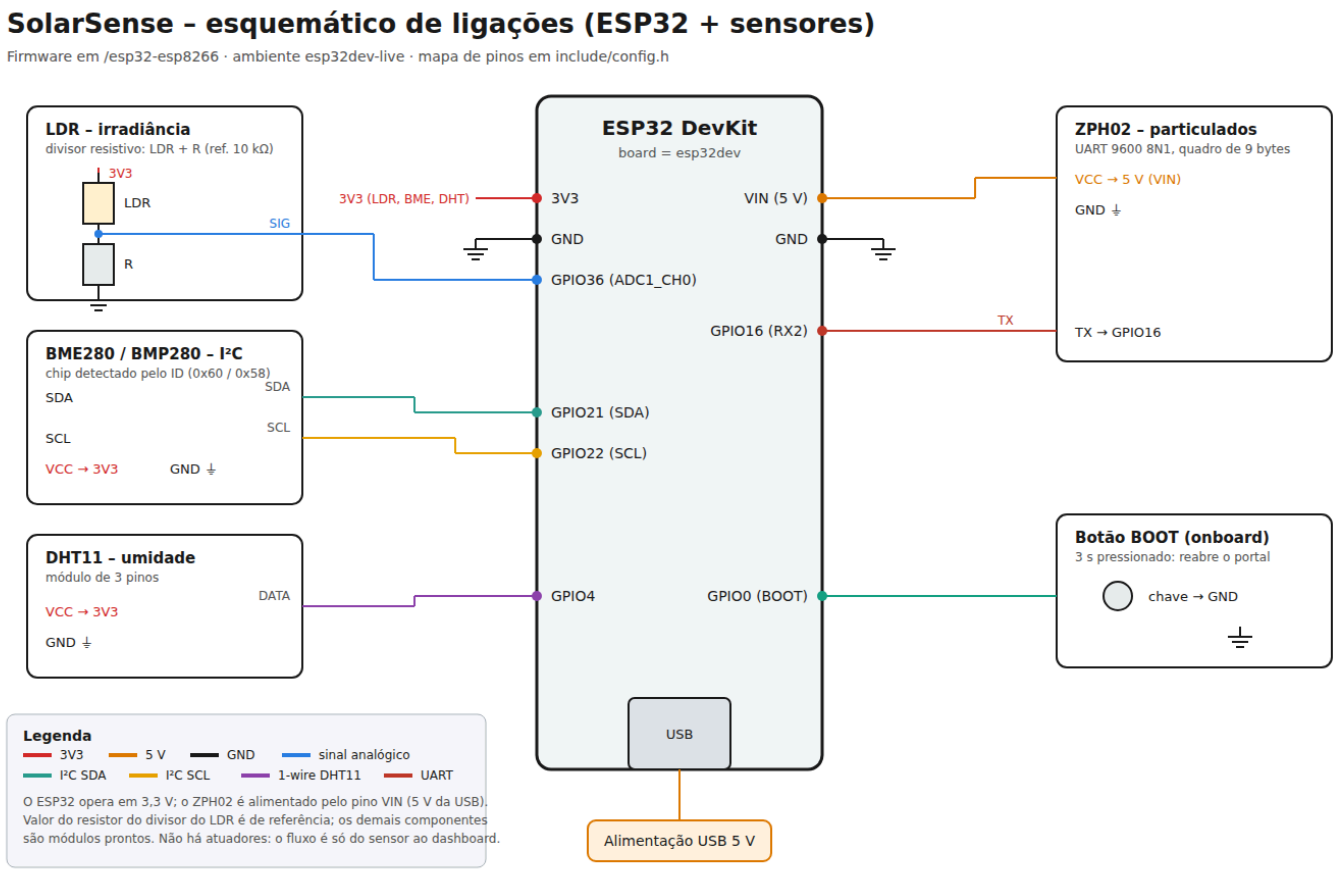
Figura 4 – Dashboard



Figura 5 – Protótipo do Front-End

Anexo A · Esquemático das ligações (ampliado)

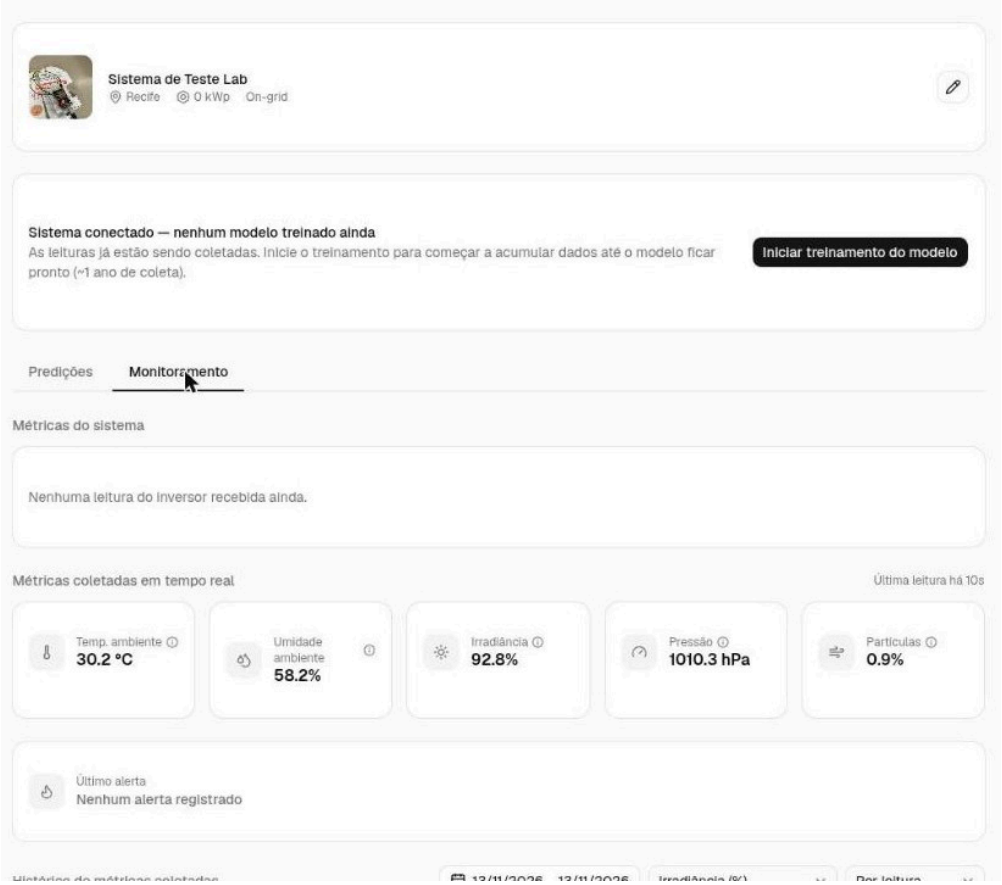
Arquivo no repositório: /schematics/solarsense-esquematico.svg



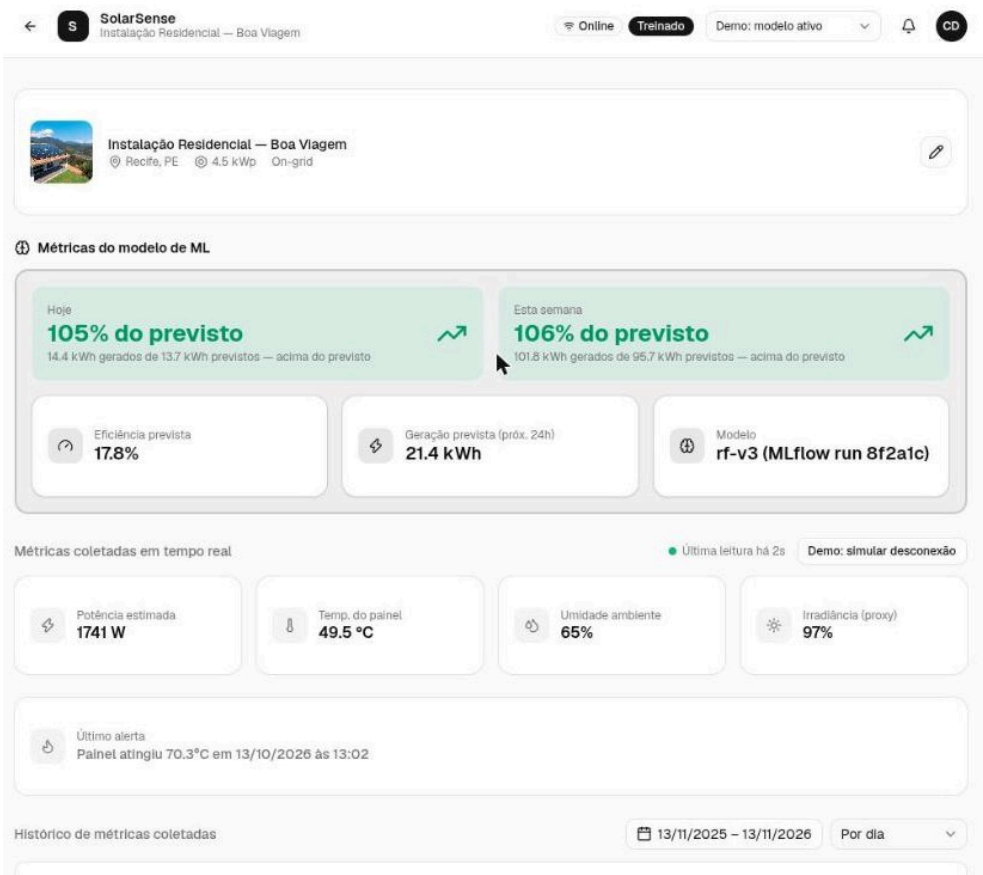
Pinos conforme esp32-esp8266/include/config.h: LDR no GPIO36, BME280/BMP280 em I²C (GPIO21/22), DHT11 no GPIO4, ZPH02 na UART2 (GPIO16) e botão BOOT no GPIO0.

Anexo B · Telas do sistema

Dashboard do app (dados em tempo real da ESP32)



Protótipo do front-end (frontend-mockup, dados fictícios)



Painel de Avaliação do Check Point

Uso exclusivo dos docentes – não preencher

Ponto avaliado	Observações dos docentes	Atende?		
Prototipagem – uso dos sensores/atuadores		Sim	Parcial	Não
Prototipagem – organização do circuito		Sim	Parcial	Não
Artefatos – códigos-fonte (Front-End, Back-End, Firmware)		Sim	Parcial	Não
Arquitetura – fluxo de comunicação		Sim	Parcial	Não
Arquitetura – infraestrutura (local/nuvem)		Sim	Parcial	Não
Documentação – Git		Sim	Parcial	Não
Aplicação – dashboard		Sim	Parcial	Não
Aplicação – protótipo do Front-End		Sim	Parcial	Não

Evolução do projeto	Adequada Parcial Insuficiente
Pontuação do Check Point	Nota: _____ / 15 pts
Parecer e encaminhamentos	
Docente avaliador / data	